

GOOGLE AI STUDIO

Agentic Coding Guide

PostgreSQL Cardinality Estimation & Query Optimization

Using Tree-LSTM + Bayesian Networks

A Complete Build-From-Scratch Manual

Target Platform: Google Colab (Free Tier) + Google AI Studio

Compute Budget: Colab T4 GPU

Scope: Course project — Deep Learning + Probabilistic Reasoning

Every command, every file, every design decision documented.

TABLE OF CONTENTS

1. PROJECT OVERVIEW & CORE CLAIM

What we build and why it matters

2. ENVIRONMENT SETUP

Google Colab + AI Studio configuration

3. PHASE 0: PROJECT SCAFFOLDING

Directory structure and base files

4. PHASE 1: SYNTHETIC DATA GENERATION

Schema design with injected correlation

5. PHASE 2: QUERY WORKLOAD GENERATION

Training and test query sets

6. PHASE 3: BAYESIAN NETWORK (OFFLINE)

Chow-Liu structure learning + CPTs

7. PHASE 4: CROSS-TABLE JOIN BRIDGE

Simplified FactorJoin via 2D contingency tables

8. PHASE 5: SQL TO AST PARSER

pglast-based query parsing pipeline

9. PHASE 6: DEEP LEARNING PIPELINE

Tree-LSTM + fusion MLP

10. PHASE 7: TRAINING LOOP

End-to-end training with diagnostics

11. PHASE 8: EVALUATION HARNESS

Q-error, plan-match, plan-sensitivity

12. GOOGLE AI STUDIO PROMPT STRATEGY

How to prompt the AI for each phase

13. COMMON FAILURE MODES & FIXES

What breaks and how to fix it

14. SUBMISSION CHECKLIST

What faculty expects to see

1. PROJECT OVERVIEW & CORE CLAIM

1.1 The Problem

PostgreSQL's query optimizer estimates result-set sizes (cardinality) using statistics that assume columns are statistically independent. When columns are correlated — especially across joined tables — these estimates can be off by orders of magnitude, causing the optimizer to choose catastrophically wrong physical plans (e.g., Nested Loop when Hash Join is optimal).

1.2 The Solution (Hybrid Architecture)

We build a **hybrid estimator** that combines two complementary approaches:

- 1. **Bayesian Network (offline, fixed)**: Learned once from real data. Captures column dependencies and conditional probabilities. Interpretable and grounded in counting.
- 2. **Tree-LSTM (online, trained)**: Encodes the structure of each incoming SQL query (its AST) into a vector. Learns how query shape affects cardinality.
- 3. **Fusion MLP**: Combines the BN's real probabilities with the Tree-LSTM's structural embedding to predict final cardinality.

1.3 Core Claim to Defend

Our hybrid estimator produces more accurate cardinality estimates than PostgreSQL's built-in estimator — measured via Q-error — specifically on queries with correlated predicates that span multiple tables, a case PostgreSQL structurally cannot address even with CREATE STATISTICS enabled.

1.4 Why This Architecture Matters Academically

This is not a pure neural approach (like NeuroCard). The BN provides interpretable, grounded probabilities; the Tree-LSTM provides learned structural reasoning. The MLP is the only component that decides how to combine them. This satisfies both courses: Deep Learning (Tree-LSTM + MLP) and Probabilistic Reasoning (BN + factor graphs).

1.5 Key Design Rules (Non-Negotiable)

- Rule 1: The Tree-LSTM NEVER sees or invents real probability numbers. It only encodes query structure.
- Rule 2: Real facts come EXCLUSIVELY from the offline BN/FactorJoin model.
- Rule 3: The MLP is the ONLY component allowed to decide how structure and facts combine.
- Rule 4: The BN/CPTs stay FIXED during training. Only the Tree-LSTM + MLP are trained end-to-end.

2. ENVIRONMENT SETUP

2.1 Platform Overview: Google AI Studio + Colab

Google AI Studio (2026) is a browser-based development environment powered by Gemini 3.5 Pro/Flash. For this project, we use it as the **agentic coding interface** — you describe what you need, and the AI generates, explains, and iterates on code. The actual execution happens in **Google Colab** (free T4 GPU tier), which provides the runtime environment for PostgreSQL, PyTorch training, and data generation.

2.2 Google AI Studio Configuration

Step 1: Access AI Studio

- Go to aistudio.google.com and sign in with your Google account.
- Select **Gemini 3.5 Pro** for complex reasoning and code generation (this project).
- Enable **Code Execution** in the tool settings panel (right sidebar).

Step 2: Project Setup in AI Studio

- Create a new project: **'CardinalityEstimation-TreeLSTM-BN'**
- In System Instructions, paste the project context (Section 12 of this guide).
- Set temperature to **0.2** (deterministic, less creative hallucination for code).
- Set max output tokens to **8192** (large code blocks need space).

2.3 Google Colab Setup

Step 1: Create Colab Notebook

- Go to colab.research.google.com
- Create new notebook: **'Cardinality_Estimation_Project.ipynb'**
- Runtime → Change runtime type → Select **GPU (T4)**

Step 2: Mount Google Drive

Mount Drive for persistent storage across sessions:

```
from google.colab import drive
drive.mount('/content/drive')
PROJECT_ROOT = '/content/drive/MyDrive/CardinalityEstimation'
```

Step 3: Install Dependencies

Run this cell once at the start of every session:

```
!pip install torch==2.3.0 torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
!pip install pgmpy==0.1.24 networkx==3.2.1 pandas==2.1.4 numpy==1.26.4
!pip install psycopg2-binary==2.9.9 sqlalchemy==2.0.23
!pip install pglast==6.2 sqlglot==23.0.0 faker==22.0.0
!pip install wandb==0.16.0 matplotlib==3.8.2 seaborn==0.13.0 scikit-learn==1.3.2
!apt-get update -qq && apt-get install -y postgresql postgresql-contrib -qq
!service postgresql start
!sudo -u postgres psql -c "CREATE USER colab_user WITH PASSWORD 'colab_pass';"
!sudo -u postgres psql -c "CREATE DATABASE cardinality_db OWNER colab_user;"
```

```
!sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE cardinality_db TO colab_user;"
```

CRITICAL: After installing PostgreSQL, you **MUST** restart the Colab runtime (Runtime → Restart runtime) before the postgres service becomes available. Then re-run only the service start and user creation commands.

2.4 Project Directory in Colab

Create this structure in your mounted Drive:

```
CardinalityEstimation/  
  ■■■ data/  
  ■ ■■■ raw/ # PostgreSQL dumps, synthetic CSVs  
  ■ ■■■ processed/ # Serialized BNs, factor graphs  
  ■ ■■■ workloads/ # train_queries.csv, test_queries.csv  
  ■■■ src/ # All Python modules  
  ■■■ notebooks/ # Development notebooks  
  ■■■ checkpoints/ # trained_model.pt files  
  ■■■ results/ # Evaluation outputs, plots
```

3. PHASE 0: PROJECT SCAFFOLDING

3.1 What the AI Must Generate First

Before any data or models, the AI must create the foundational files. These are not optional — they define the contract between components. Generate them in this exact order:

File 1: config.yaml

Central configuration. ALL hyperparameters, paths, and schema definitions live here. No hardcoded values anywhere else.

WHY THIS MATTERS: Every number in this config is a measured parameter, not an illustration. The correlation strengths are KNOWN because we inject them during data generation. This is how you validate the BN.

File 2: requirements.txt

Pinned versions prevent Colab environment drift:

```
torch==2.3.0
torchvision==0.18.0
torchaudio==2.3.0
pgmpy==0.1.24
networkx==3.2.1
pandas==2.1.4
numpy==1.26.4
psycogp2-binary==2.9.9
sqlalchemy==2.0.23
pglast==6.2
sqlglot==23.0.0
faker==22.0.0
wandb==0.16.0
matplotlib==3.8.2
seaborn==0.13.0
scikit-learn==1.3.2
pyyaml==6.0.1
tqdm==4.66.1
```

File 3: src/utils/config_loader.py

Create a config loader used by every module:

```
import yaml
import os
from pathlib import Path

class Config:
    def __init__(self, config_path='/content/drive/MyDrive/CardinalityEstimation/config.yaml'):
        with open(config_path, 'r') as f:
            self._config = yaml.safe_load(f)
        self.project_root = Path(self._config['project']['root_dir'])
        self.project_root.mkdir(parents=True, exist_ok=True)

    def __getitem__(self, key):
        return self._config[key]

    def get(self, key, default=None):
        return self._config.get(key, default)

    def db_connection_string(self):
```

```

db = self._config['database']
return f"postgresql://{db['user']}:{db['password']}@{db['host']}:{db['port']}/{db['name']}"

_config_instance = None

def get_config():
    global _config_instance
    if _config_instance is None:
        _config_instance = Config()
    return _config_instance

```

File 4: src/utlils/db.py

Database connection utility:

```

import psycopg2
from psycopg2.extras import RealDictCursor
from .config_loader import get_config

class Database:
    def __init__(self):
        self.config = get_config()
        self.conn = None

    def connect(self):
        db = self.config['database']
        self.conn = psycopg2.connect(
            host=db['host'], port=db['port'],
            database=db['name'], user=db['user'], password=db['password']
        )
        return self.conn

    def execute(self, query, params=None, fetch=True):
        if self.conn is None:
            self.connect()
        with self.conn.cursor(cursor_factory=RealDictCursor) as cur:
            cur.execute(query, params)
            if fetch:
                return cur.fetchall()
            self.conn.commit()

    def close(self):
        if self.conn:
            self.conn.close()
        self.conn = None

```

GOOGLE AI STUDIO PROMPT for Phase 0:

'Generate the complete project scaffolding for a PostgreSQL cardinality estimation project. Create config.yaml with synthetic schema definitions for 3 tables (employees, projects, assignments), correlation injection parameters (same-table: dept-city at 0.85, cross-table: city-budget_tier at 0.60), model hyperparameters, and evaluation metrics. Also create requirements.txt with pinned versions, and base utility files for config loading and database connection. All paths must use the Google Drive mount at /content/drive/MyDrive/CardinalityEstimation. Include negative control independent pairs (salary_tier, experience_years). Every parameter must be real and measured, not illustrative.'

4. PHASE 1: SYNTHETIC DATA GENERATION

4.1 Why Synthetic Data with Known Correlation

You CANNOT validate a correlation detector on a dataset where you don't know the ground-truth correlation. By generating data with controlled, quantified correlation, you create an oracle: if your BN says $P(\text{city=Bangalore} \mid \text{dept=Engineering}) = 0.83$, and you injected 0.85, you know the BN learned correctly. This is how you hand-validate.

4.2 The Data Generator Architecture

The generator must produce 3 artifacts:

- 1. Independent base columns: Generated from marginal distributions (uniform or skewed).
- 2. Same-table correlated columns: Generated conditionally on the independent base (e.g., city depends on dept).
- 3. Cross-table correlated columns: Generated by propagating correlation through join keys (e.g., projects.budget_tier depends on employees.city via the dept join key).

4.3 Implementation: src/schema/data_generator.py

This is the MOST CRITICAL file. Every bug here invalidates your entire evaluation.

```
import numpy as np
import pandas as pd
from faker import Faker
from pathlib import Path
import yaml
from typing import Dict

class CorrelatedDataGenerator:
    def __init__(self, config_path='/content/drive/MyDrive/CardinalityEstimation/config.yaml'):
        with open(config_path, 'r') as f:
            self.config = yaml.safe_load(f)
            self.schema = self.config['schema']
            self.corr_config = self.config['correlation']
            self.rng = np.random.RandomState(self.config['project']['seed'])
            self.fake = Faker()
            Faker.seed(self.config['project']['seed'])

    def generate_table(self, table_name: str) -> pd.DataFrame:
        table_cfg = self.schema['tables'][table_name]
        n_rows = table_cfg['num_rows']
        df = pd.DataFrame()

        # Step 1: Generate independent base columns first
        independent_cols = self._get_independent_columns(table_name)
        for col in table_cfg['columns']:
            col_name = col['name']
            if col_name in independent_cols:
                df[col_name] = self._generate_independent_column(col, n_rows)

        # Step 2: Generate same-table correlated columns
        same_table_corr = self._get_same_table_correlation(table_name)
        if same_table_corr:
            base_col = same_table_corr['columns'][0]
            target_col = same_table_corr['columns'][1]
            df[target_col] = self._generate_correlated_column(
                df[base_col], same_table_corr, n_rows
            )
```

```

# Step 3: Generate remaining independent columns
for col in table_cfg['columns']:
    col_name = col['name']
    if col_name not in df.columns:
        df[col_name] = self._generate_independent_column(col, n_rows)

return df

def _generate_independent_column(self, col_cfg: dict, n_rows: int):
    if 'categories' in col_cfg:
        probs = [1.0 / len(col_cfg['categories'])] * len(col_cfg['categories'])
        return self.rng.choice(col_cfg['categories'], size=n_rows, p=probs)
    elif 'range' in col_cfg:
        low, high = col_cfg['range']
        return self.rng.randint(low, high + 1, size=n_rows)
    else:
        return [self.fake.word() for _ in range(n_rows)]

def _generate_correlated_column(self, base_series: pd.Series,
                                corr_cfg: dict, n_rows: int) -> pd.Series:
    conditional_probs = corr_cfg['conditional_probs']
    result = []
    for val in base_series:
        val_str = str(val)
        if val_str in conditional_probs:
            choices = list(conditional_probs[val_str].keys())
            probs = list(conditional_probs[val_str].values())
            probs = np.array(probs) / sum(probs)
            result.append(self.rng.choice(choices, p=probs))
        else:
            result.append(self.rng.choice(list(conditional_probs.values())[0].keys()))
    return pd.Series(result)

def generate_cross_table_correlation(self, employees_df: pd.DataFrame,
                                     projects_df: pd.DataFrame) -> pd.DataFrame:
    cross_corr = self.corr_config['cross_table'][0]
    conditional_probs = cross_corr['conditional_probs']

    for idx, row in projects_df.iterrows():
        dept = row['dept']
        emp_cities = employees_df[employees_df['dept'] == dept]['city'].values
        if len(emp_cities) > 0:
            city_counts = pd.Series(emp_cities).value_counts(normalize=True)
            weighted_probs = {}
            for city, city_prob in city_counts.items():
                if city in conditional_probs:
                    for tier, tier_prob in conditional_probs[city].items():
                        tier_clean = int(tier.replace('budget_tier_', ''))
                        weighted_probs[tier_clean] = weighted_probs.get(tier_clean, 0) + city_prob * tier_prob

            total = sum(weighted_probs.values())
            if total > 0:
                for k in weighted_probs:
                    weighted_probs[k] /= total
                tiers = list(weighted_probs.keys())
                probs = list(weighted_probs.values())
                projects_df.at[idx, 'budget_tier'] = self.rng.choice(tiers, p=probs)

    return projects_df

def generate_all(self) -> Dict[str, pd.DataFrame]:
    employees = self.generate_table('employees')
    projects = self.generate_table('projects')

```

```

projects = self.generate_cross_table_correlation(employees, projects)
assignments = self.generate_table('assignments')

valid_emp_ids = set(employees['emp_id'].values)
valid_proj_ids = set(projects['proj_id'].values)
assignments = assignments[
    assignments['emp_id'].isin(valid_emp_ids) &
    assignments['proj_id'].isin(valid_proj_ids)
]

return {'employees': employees, 'projects': projects, 'assignments': assignments}

def save_to_postgres(self, dfs: Dict[str, pd.DataFrame]):
    from sqlalchemy import create_engine
    from ..utils.config_loader import get_config

    config = get_config()
    engine = create_engine(config.db_connection_string())

    for table_name, df in dfs.items():
        df.to_sql(table_name, engine, if_exists='replace', index=False)
        print(f"Loaded {len(df)} rows into {table_name}")

    with engine.connect() as conn:
        conn.execute("ALTER TABLE employees ADD PRIMARY KEY (emp_id)")
        conn.execute("ALTER TABLE projects ADD PRIMARY KEY (proj_id)")
        conn.execute("ALTER TABLE assignments ADD PRIMARY KEY (assign_id)")
        conn.execute("ALTER TABLE assignments ADD CONSTRAINT fk_emp FOREIGN KEY (emp_id) REFERENCES
employees(emp_id), ADD CONSTRAINT fk_proj FOREIGN KEY (proj_id) REFERENCES projects(proj_id)")
        conn.commit()

```

4.4 Validation: How to Prove Correlation Was Injected

After generation, run this validation script:

```

import pandas as pd
from sqlalchemy import create_engine
from src.utils.config_loader import get_config

config = get_config()
engine = create_engine(config.db_connection_string())

# Validate same-table correlation: P(city | dept=Engineering)
df = pd.read_sql("SELECT dept, city FROM employees", engine)
eng_cities = df[df['dept'] == 'Engineering']['city'].value_counts(normalize=True)
print("P(city | Engineering):")
print(eng_cities)
print(f"Bangalore proportion: {eng_cities.get('Bangalore', 0):.3f} (expected: 0.850)")

# Validate cross-table correlation via join
query = "SELECT e.city, p.budget_tier, COUNT(1) as cnt FROM employees e JOIN projects p ON e.dept = p.dept
GROUP BY e.city, p.budget_tier"
joint = pd.read_sql(query, engine)
pivot = joint.pivot(index='city', columns='budget_tier', values='cnt').fillna(0)
pivot = pivot.div(pivot.sum(axis=1), axis=0)
print("P(budget_tier | city) from actual join:")
print(pivot)
print(f"P(budget_tier=5 | Bangalore): {pivot.loc['Bangalore', 5]:.3f} (expected: ~0.600)")

```

GOOGLE AI STUDIO PROMPT for Phase 1:

'Implement the complete synthetic data generator for a PostgreSQL cardinality estimation project. The generator must create 3 tables (employees, projects, assignments) with KNOWN, QUANTIFIED correlation. Inject same-table correlation:

85% of Engineering employees are in Bangalore. Inject cross-table correlation: projects in the same dept as Bangalore employees have 60% probability of budget_tier=5. Include a validation script that computes the ACTUAL conditional probabilities from the generated data and compares them to the injected values. Use the config.yaml schema I provided. Save all data to PostgreSQL. CRITICAL: The validation must prove the correlation was actually injected — this is my ground truth for the entire project.'

5. PHASE 2: QUERY WORKLOAD GENERATION

5.1 What the Workload Must Contain

The workload is your training and test data. Each query must have: (1) SQL text, (2) true cardinality from EXPLAIN ANALYZE, (3) Postgres's own estimate from EXPLAIN, (4) which physical plan Postgres chose. Without (3) and (4), you cannot compare against the baseline.

5.2 Query Categories (Must Be Balanced)

- Category A — Single-table, independent predicates (40%): `SELECT ... FROM employees WHERE salary_tier=3 AND experience_years>5`
- Category B — Single-table, correlated predicates (15%): `SELECT ... FROM employees WHERE dept='Engineering' AND city='Bangalore'`
- Category C — Two-table join, independent predicates (25%): `SELECT ... FROM employees e JOIN projects p ON e.dept=p.dept WHERE e.salary_tier=3 AND p.priority='High'`
- Category D — Two-table join, cross-table correlated predicates (15%): `SELECT ... FROM employees e JOIN projects p ON e.dept=p.dept WHERE e.city='Bangalore' AND p.budget_tier=5`
- Category E — Three-table join, mixed (5%): `SELECT ... FROM employees e JOIN projects p ON e.dept=p.dept JOIN assignments a ON e.emp_id=a.emp_id WHERE ...`

5.3 Implementation: `src/workload/generator.py`

```
import random
import numpy as np
import pandas as pd
from typing import List, Dict, Tuple
from ..utils.config_loader import get_config
from ..utils.db import Database

class QueryWorkloadGenerator:
    def __init__(self):
        self.config = get_config()
        self.schema = self.config['schema']
        self.db = Database()
        self.rng = np.random.RandomState(self.config['project']['seed'])
        self.column_values = self._load_column_values()

    def _load_column_values(self) -> Dict:
        values = {}
        for table_name in self.schema['tables'].keys():
            values[table_name] = {}
            for col in self.schema['tables'][table_name]['columns']:
                col_name = col['name']
                if col_name.endswith('_id'):
                    continue
                result = self.db.execute(f"SELECT DISTINCT {col_name} FROM {table_name} LIMIT 100")
                values[table_name][col_name] = [row[col_name] for row in result]
            return values

    def generate_predicate(self, table_name: str, correlated: bool = False) -> Tuple[str, str]:
        if correlated and table_name == 'employees':
            dept_val = self.rng.choice(self.column_values['employees']['dept'])
            city_val = self.rng.choice(self.column_values['employees']['city'])
            return f"dept = '{dept_val}' AND city = '{city_val}'", True
```

```

col = self.rng.choice([c for c in self.schema['tables'][table_name]['columns']
if not c['name'].endswith('_id')])
col_name = col['name']
val = self.rng.choice(self.column_values[table_name][col_name])

if 'range' in col:
op = self.rng.choice(['=', '>', '<', '>=', '<='])
return f"{col_name} {op} {val}", False
else:
op = self.rng.choice(['=', '!='])
return f"{col_name} {op} '{val}'", False

def generate_single_table_query(self, table_name: str, num_predicates: int,
force_correlated: bool = False) -> str:
predicates = []
has_correlated = False

if force_correlated and table_name == 'employees':
pred, _ = self.generate_predicate(table_name, correlated=True)
predicates.append(pred)
has_correlated = True
remaining = num_predicates - 2
else:
remaining = num_predicates

for _ in range(remaining):
pred, is_corr = self.generate_predicate(table_name, correlated=False)
predicates.append(pred)

where_clause = ' AND '.join(predicates)
return f"SELECT COUNT(1) FROM {table_name} WHERE {where_clause}", has_correlated

def generate_join_query(self, tables: List[str], num_predicates: int,
force_cross_correlated: bool = False) -> str:
join_conditions = []
for i in range(len(tables) - 1):
t1, t2 = tables[i], tables[i+1]
shared = self._find_join_key(t1, t2)
join_conditions.append(f"{t1}.{shared} = {t2}.{shared}")

predicates = []
if force_cross_correlated and 'employees' in tables and 'projects' in tables:
city_val = self.rng.choice(self.column_values['employees']['city'])
tier_val = self.rng.choice(self.column_values['projects']['budget_tier'])
predicates.append(f"employees.city = '{city_val}'")
predicates.append(f"projects.budget_tier = {tier_val}")
remaining = num_predicates - 2
else:
remaining = num_predicates

for _ in range(remaining):
table = self.rng.choice(tables)
pred, _ = self.generate_predicate(table, correlated=False)
predicates.append(pred)

select = f"SELECT COUNT(1) FROM {tables[0]}"
for i, t in enumerate(tables[1:]):
select += f" JOIN {t} ON {join_conditions[i]}"

where_clause = ' AND '.join(predicates)
return f"{select} WHERE {where_clause}", force_cross_correlated

def _find_join_key(self, t1: str, t2: str) -> str:
if 'employees' in [t1, t2] and 'projects' in [t1, t2]:

```

```

return 'dept'
return 'emp_id'

def generate_workload(self, size: int, split: str = 'train') -> pd.DataFrame:
    queries = []
    dist = self.config['workload']['predicate_distribution']
    corr_ratio = self.config['workload']['correlation_query_ratio']

    for i in range(size):
        category_roll = self.rng.random()
        force_corr = self.rng.random() < corr_ratio

        if category_roll < dist['single_table']:
            table = self.rng.choice(list(self.schema['tables'].keys()))
            num_pred = self.rng.randint(1, self.config['workload']['max_predicates'] + 1)
            sql, is_corr = self.generate_single_table_query(table, num_pred, force_corr)
            queries.append({'sql': sql, 'type': 'single', 'tables': [table],
                          'is_correlated': is_corr, 'num_predicates': num_pred})

        elif category_roll < dist['single_table'] + dist['two_table_join']:
            tables = self.rng.choice(['employees', 'projects', 'assignments'], size=2, replace=False)
            num_pred = self.rng.randint(1, self.config['workload']['max_predicates'] + 1)
            is_cross = 'employees' in tables and 'projects' in tables
            force_cross = force_corr and is_cross
            sql, is_corr = self.generate_join_query(list(tables), num_pred, force_cross)
            queries.append({'sql': sql, 'type': 'join2', 'tables': list(tables),
                          'is_correlated': is_corr, 'num_predicates': num_pred})

        else:
            tables = ['employees', 'projects', 'assignments']
            num_pred = self.rng.randint(1, self.config['workload']['max_predicates'] + 1)
            sql, is_corr = self.generate_join_query(tables, num_pred, force_corr)
            queries.append({'sql': sql, 'type': 'join3', 'tables': tables,
                          'is_correlated': is_corr, 'num_predicates': num_pred})

    return pd.DataFrame(queries)

def label_queries(self, df: pd.DataFrame) -> pd.DataFrame:
    results = []
    for _, row in df.iterrows():
        sql = row['sql']

        explain_analyze = self.db.execute(f"EXPLAIN ANALYZE {sql}")
        true_card = self._extract_cardinality_from_explain(explain_analyze, actual=True)

        explain = self.db.execute(f"EXPLAIN {sql}")
        pg_estimate = self._extract_cardinality_from_explain(explain, actual=False)
        pg_plan = self._extract_plan_type(explain)

        row_dict = row.to_dict()
        row_dict['true_cardinality'] = true_card
        row_dict['postgres_estimate'] = pg_estimate
        row_dict['postgres_plan'] = pg_plan
        results.append(row_dict)

    return pd.DataFrame(results)

def _extract_cardinality_from_explain(self, explain_rows, actual: bool = False):
    import re
    for row in explain_rows:
        line = row.get('QUERY PLAN', '')
        if actual and 'actual rows=' in line:
            match = re.search(r'actual rows=(\d+)', line)
            if match:

```

```

return int(match.group(1))
elif not actual and 'rows=' in line and 'actual' not in line:
    match = re.search(r'rows=(\d+)', line)
    if match:
        return int(match.group(1))
    return -1

def _extract_plan_type(self, explain_rows):
    for row in explain_rows:
        line = row.get('QUERY PLAN', '')
        if 'Nested Loop' in line:
            return 'Nested Loop'
        elif 'Hash Join' in line:
            return 'Hash Join'
        elif 'Seq Scan' in line:
            return 'Seq Scan'
        elif 'Index Scan' in line:
            return 'Index Scan'
        return 'Unknown'

```

5.4 Critical Implementation Notes

- EXPLAIN (without ANALYZE) gives Postgres's estimate WITHOUT executing the query. This is your baseline to beat.
- EXPLAIN ANALYZE executes the query and reports actual rows. This is your ground truth label.
- The difference between these two numbers IS the problem you're solving.
- For 10,000 queries, EXPLAIN ANALYZE will take time. Run overnight or in batches. Save intermediate results every 100 queries.

GOOGLE AI STUDIO PROMPT for Phase 2:

'Implement a query workload generator that produces 8000 training and 2000 test SQL queries for a PostgreSQL cardinality estimation project. The generator must create 5 categories: single-table independent, single-table correlated, two-table join independent, two-table join cross-correlated, and three-table join. For each query, run EXPLAIN to get Postgres's estimate and EXPLAIN ANALYZE to get true cardinality. Also extract which physical plan Postgres chose (Nested Loop, Hash Join, Seq Scan, Index Scan). Save as CSV with columns: sql, type, tables, is_correlated, num_predicates, true_cardinality, postgres_estimate, postgres_plan. CRITICAL: The cross-correlated queries MUST target the known injected correlation (employees.city + projects.budget_tier). Include 50% correlated queries and 50% independent as negative controls.'

6. PHASE 3: BAYESIAN NETWORK (OFFLINE)

6.1 What the BN Must Learn

For each table, the BN learns: (1) Structure — which columns depend on which (via Chow-Liu algorithm), and (2) Parameters — Conditional Probability Tables (CPTs) via Maximum Likelihood Estimation (counting).

6.2 Chow-Liu Algorithm Steps

- 1. Compute mutual information $I(X;Y)$ for every pair of columns
- 2. Build a complete graph with MI as edge weights
- 3. Find Maximum Spanning Tree (MST) — this is the BN structure
- 4. Pick any node as root, direct edges away from root

6.3 Implementation: src/bayesian/chow_liu.py

```
import numpy as np
import pandas as pd
from pgmpy.models import BayesianNetwork
from pgmpy.estimators import TreeSearch, MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination
import pickle
from ..utils.config_loader import get_config
from ..utils.db import Database

class PerTableBN:
    def __init__(self, table_name: str):
        self.config = get_config()
        self.table_name = table_name
        self.model = None
        self.inference = None
        self.db = Database()

    def learn_structure(self):
        df = pd.read_sql(f"SELECT * FROM {self.table_name}", con=self.db.conn)

        # Drop ID columns
        id_cols = [c['name'] for c in self.config['schema']['tables'][self.table_name]['columns']
        if c['name'].endswith('_id')]
        df = df.drop(columns=id_cols, errors='ignore')

        # Discretize continuous columns
        for col in df.columns:
            if df[col].dtype in ['int64', 'float64']:
                df[col] = pd.qcut(df[col], q=10, labels=False, duplicates='drop')

        # Chow-Liu structure learning
        est = TreeSearch(df, root_node=df.columns[0])
        dag = est.estimate(estimator_type='chow-liu', class_node=None)

        self.model = BayesianNetwork(dag.edges())
        return self

    def learn_parameters(self):
        df = pd.read_sql(f"SELECT * FROM {self.table_name}", con=self.db.conn)
        id_cols = [c['name'] for c in self.config['schema']['tables'][self.table_name]['columns']
        if c['name'].endswith('_id')]
        df = df.drop(columns=id_cols, errors='ignore')
```

```

for col in df.columns:
    if df[col].dtype in ['int64', 'float64']:
        df[col] = pd.qcut(df[col], q=10, labels=False, duplicates='drop')

# MLE with Laplace smoothing
self.model.fit(df, estimator=MaximumLikelihoodEstimator)
self.inference = VariableElimination(self.model)
return self

def query_probability(self, evidence: dict) -> float:
    if self.inference is None:
        raise ValueError("Model not trained")
    result = self.inference.query(
        variables=list(self.model.nodes() - set(evidence.keys())),
        evidence=evidence,
        show_progress=False
    )
    return float(result.values.sum())

def save(self, path: str):
    with open(path, 'wb') as f:
        pickle.dump({'model': self.model, 'inference': self.inference}, f)

@classmethod
def load(cls, path: str):
    with open(path, 'rb') as f:
        data = pickle.load(f)
    instance = cls.__new__(cls)
    instance.model = data['model']
    instance.inference = data['inference']
    return instance

```

6.4 Hand-Validation Against Known Correlation

This is how you prove the BN learned correctly:

```

def validate_bn_against_injected_correlation(bn_path: str, table_name: str):
    bn = PerTableBN.load(bn_path)

    # Test 1: P(city=Bangalore | dept=Engineering) should be ~0.85
    prob = bn.inference.query(
        variables=['city'],
        evidence={'dept': 'Engineering'},
        show_progress=False
    )
    bangalore_prob = prob.get_value(city='Bangalore')
    print(f"P(city=Bangalore | dept=Engineering) = {bangalore_prob:.3f} (expected: 0.850)")
    assert 0.80 <= bangalore_prob <= 0.90, "BN failed to learn injected correlation!"

    # Test 2: P(city=Delhi | dept=Sales) should be ~0.50
    prob = bn.inference.query(
        variables=['city'],
        evidence={'dept': 'Sales'},
        show_progress=False
    )
    delhi_prob = prob.get_value(city='Delhi')
    print(f"P(city=Delhi | dept=Sales) = {delhi_prob:.3f} (expected: 0.500)")
    assert 0.45 <= delhi_prob <= 0.55, "BN failed on second correlation test!"

    print("BN validation PASSED")

```

GOOGLE AI STUDIO PROMPT for Phase 3:

'Implement per-table Bayesian Network learning using Chow-Liu structure learning and Maximum Likelihood Estimation with Laplace smoothing. For each table (employees, projects, assignments): load data from PostgreSQL, discretize continuous columns into 10 bins, learn the tree structure via mutual information, then fill CPTs by counting. Include a validation script that queries the learned BN for $P(\text{city}|\text{dept}=\text{Engineering})$ and proves it matches the injected correlation (~ 0.85). Save serialized BN objects to disk. CRITICAL: The BN must be queryable offline — no database access at inference time.'

7. PHASE 4: CROSS-TABLE JOIN BRIDGE (SIMPLIFIED)

7.1 Why NOT Full FactorJoin

Full FactorJoin requires: complex factor graph construction, progressive estimation across overlapping subqueries, and sophisticated join key handling. This is 2+ weeks of engineering. We implement the 2D contingency table fallback — still a valid factor graph with one factor node.

7.2 The Simplified Approach

For employees JOIN projects ON dept:

- 1. Bin the join key 'dept' (it's already categorical, so bins = categories)
- 2. Build 2D contingency table: COUNT(*) GROUP BY employees.city, projects.budget_tier, employees.dept
- 3. At query time: $P(\text{city}=\text{'Bangalore'} \text{ AND } \text{budget_tier}=5 \mid \text{dept match}) = \text{count}(\text{Bangalore}, 5, \text{dept}) / \text{count}(\text{all dept rows})$

7.3 Implementation: src/join/contingency_table.py

```
import pandas as pd
import numpy as np
from ..utils.config_loader import get_config
from ..utils.db import Database

class ContingencyTableJoinBridge:
    def __init__(self, table1: str, table2: str, join_key: str):
        self.t1 = table1
        self.t2 = table2
        self.join_key = join_key
        self.config = get_config()
        self.db = Database()
        self.contingency = None
        self.conditional_probs = None

    def build(self):
        query = f"""SELECT
            t1.{self.join_key} AS join_val,
            t1.city AS t1_city,
            t2.budget_tier AS t2_budget_tier,
            COUNT(1) AS cnt
        FROM {self.t1} t1
        JOIN {self.t2} t2
        ON t1.{self.join_key} = t2.{self.join_key}
        GROUP BY
            t1.{self.join_key},
            t1.city,
            t2.budget_tier"""

        df = pd.read_sql(query, self.db.conn)

        self.contingency = df.pivot_table(
            index=["join_val", "t1_city"],
            columns="t2_budget_tier",
            values="cnt",
            fill_value=0
        )

        self.conditional_probs = self.contingency.div(
```

```

        self.contingency.sum(axis=1),
        axis=0
    )

    return self

def query_probability(
    self,
    join_val,
    t1_predicate: dict,
    t2_predicate: dict
) -> float:

    t1_col = list(t1_predicate.keys())[0]
    t1_val = list(t1_predicate.values())[0]

    t2_col = list(t2_predicate.keys())[0]
    t2_val = list(t2_predicate.values())[0]

    key = (join_val, t1_val)

    if key not in self.conditional_probs.index:
        return 1.0 / len(self.conditional_probs.columns)

    if t2_val not in self.conditional_probs.columns:
        return 0.0

    prob = self.conditional_probs.loc[key, t2_val]

    return float(prob) if not np.isnan(prob) else 0.0

def save(self, path: str):
    self.conditional_probs.to_pickle(path)

@classmethod
def load(cls, path: str):
    instance = cls.__new__(cls)
    instance.conditional_probs = pd.read_pickle(path)
    return instance

```

7.4 Validation

```

def validate_join_bridge(bridge: ContingencyTableJoinBridge):
    prob = bridge.query_probability(
        join_val="Engineering",
        t1_predicate={"city": "Bangalore"},
        t2_predicate={"budget_tier": 5}
    )

    print(
        f"P(budget_tier=5 | Engineering, Bangalore) = "
        f"{prob:.3f} (expected: ~0.600)"
    )

    assert 0.50 <= prob <= 0.70, (
        "Join bridge failed validation!"
    )

```

GOOGLE AI STUDIO PROMPT for Phase 4:

'Implement a simplified cross-table join bridge for cardinality estimation. Instead of full FactorJoin with variable elimination, use a 2D contingency table approach: for employees JOIN projects ON dept, build a pivot table of COUNT(*) GROUP BY dept, employees.city, projects.budget_tier. Normalize to get P(budget_tier | dept, city). Include validation that

$P(\text{budget_tier}=5 \mid \text{dept}=\text{Engineering}, \text{city}=\text{Bangalore})$ matches the injected correlation. Save the conditional probability table to disk. Cite FactorJoin as design lineage but explicitly state this is a constrained special case.'

8. PHASE 5: SQL TO AST PARSER

8.1 Why pglast

pglast wraps PostgreSQL's actual parser — your tree matches what Postgres sees. This guarantees consistency between your model and the optimizer.

8.2 Implementation: src/parser/pglast_parser.py

```
import pglast
from pglast import ast
from typing import Any, Dict, Optional

class QueryASTNode:
    def __init__(
        self,
        node_type: str,
        value: Optional[Dict[str, Any]] = None,
        children=None
    ):
        self.node_type = node_type
        self.value = value
        self.children = children or []

    def __repr__(self):
        if self.node_type == "PREDICATE":
            return f"PRED({self.value})"

        return (
            f"{self.node_type}("
            f"{[repr(child) for child in self.children]}"
            f")"
        )

class SQLToAST:
    OPERATOR_MAP = {
        "AEXPR_OP": "=",
        "AEXPR_IN": "IN",
        "AEXPR_NOT_IN": "NOT IN",
        "AEXPR_LIKE": "LIKE",
        "AEXPR_ILIKE": "ILIKE",
        "AEXPR_NOT_LIKE": "NOT LIKE",
        "AEXPR_NOT_ILIKE": "NOT ILIKE",
    }

    def __init__(self, config=None):
        self.config = config or {
            "schema": {
                "tables": {}
            }
        }

        self.column_embeddings = {}

        self.operator_embeddings = {
            "=": 0,
            "!=": 1,
            ">": 2,
            "<": 3,
```

```

        ">=": 4,
        "<=": 5,
        "LIKE": 6,
        "IN": 7
    }

def parse(self, sql: str) -> QueryASTNode:
    tree = pglast.parse_sql(sql)

    if not tree:
        raise ValueError("SQL query produced an empty parse tree.")

    return self._convert_pglast_tree(tree)

def _convert_pglast_tree(self, tree) -> QueryASTNode:
    stmt = tree[0].stmt

    if hasattr(stmt, "whereClause") and stmt.whereClause:
        root = self._parse_where_clause(stmt.whereClause)

        # Preserve JOIN information when possible.
        if hasattr(stmt, "fromClause") and stmt.fromClause:
            join_nodes = self._extract_joins(stmt.fromClause)

            if join_nodes:
                return QueryASTNode(
                    "JOIN",
                    children=[root] + join_nodes
                )

        return root

    return QueryASTNode(
        "PREDICATE",
        value={
            "column": None,
            "op": None,
            "value": None,
            "table": None
        }
    )

def _parse_where_clause(self, node) -> QueryASTNode:

    # AND / OR are represented as BoolExpr nodes in PostgreSQL.
    if isinstance(node, ast.BoolExpr):

        if node.boolop == ast.BoolExprType.AND_EXPR:
            children = [
                self._parse_where_clause(arg)
                for arg in node.args
            ]

            return QueryASTNode(
                "AND",
                children=children
            )

        elif node.boolop == ast.BoolExprType.OR_EXPR:
            children = [
                self._parse_where_clause(arg)
                for arg in node.args
            ]

            return QueryASTNode(

```

```

        "OR",
        children=children
    )

elif node.boolop == ast.BoolExprType.NOT_EXPR:
    child = self._parse_where_clause(node.args[0])

    return QueryASTNode(
        "NOT",
        children=[child]
    )

# Standard binary expressions.
if isinstance(node, ast.A_Expr):

    column = self._extract_column(node.lexpr)
    operator = self._extract_operator(node)
    value = self._extract_value(node.rexpr)

    return QueryASTNode(
        "PREDICATE",
        value={
            "column": column,
            "op": operator,
            "value": value,
            "table": self._infer_table(column)
        }
    )

# Fallback.
return QueryASTNode(
    "PREDICATE",
    value={
        "column": None,
        "op": None,
        "value": None,
        "table": None
    }
)

def _extract_column(self, node) -> str:

    if isinstance(node, ast.ColumnRef):
        return ".".join(
            field.sval if hasattr(field, "sval")
            else field.str
            for field in node.fields
        )

    return str(node)

def _extract_operator(self, node) -> str:

    if hasattr(node, "name") and node.name:
        return node.name[0].sval

    if hasattr(node, "kind"):
        kind_name = str(node.kind)

        for key, value in self.OPERATOR_MAP.items():
            if key in kind_name:
                return value

    return "="

```

```

def _extract_value(self, node):

    if isinstance(node, ast.A_Const):

        if hasattr(node.val, "ival"):
            return node.val.ival

        if hasattr(node.val, "fval"):
            return float(node.val.fval)

        if hasattr(node.val, "sval"):
            return node.val.sval

    return str(node)

def _infer_table(self, column: str) -> str:

    # Table-qualified column:
    # employees.city
    if "." in column:
        return column.split(".")[0]

    column_name = column.split(".")[-1]

    for table, columns in self.config["schema"]["tables"].items():

        for col in columns:

            if isinstance(col, dict):
                if col.get("name") == column_name:
                    return table

            elif col == column_name:
                return table

    return "unknown"

def _extract_joins(self, from_clause):

    joins = []

    for item in from_clause:

        if isinstance(item, ast.JoinExpr):

            join_condition = None

            if itemquals:
                join_condition = self._parse_where_clause(
                    itemquals
                )

            if join_condition:
                joins.append(join_condition)

    return joins

```

8.3 Parser Test

```

def test_sql_parser():

    sql = """
    SELECT e.id, p.project_name
    FROM employees e

```

```

JOIN projects p
  ON e.dept = p.dept
WHERE e.city = 'Bangalore'
      AND p.budget_tier = 5;
"""

parser = SQLToAST()

ast_root = parser.parse(sql)

print("Parsed AST:")
print(ast_root)

assert ast_root is not None

# The WHERE clause should contain two predicates.
predicates = []

def collect(node):
    if node.node_type == "PREDICATE":
        predicates.append(node.value)

    for child in node.children:
        collect(child)

collect(ast_root)

assert len(predicates) >= 2

print("Parser test passed.")

```

GOOGLE AI STUDIO PROMPT for Phase 5:

'Implement a SQL to AST parser using pglast (PostgreSQL's native parser wrapper). Parse SELECT queries with WHERE clauses containing AND/OR combinations of predicates. Normalize the AST to a tree with node types: AND, OR, JOIN, PREDICATE. Each PREDICATE leaf should contain: column name, operator, value, and inferred table. Handle table-qualified columns such as employees.city. Include a test that parses a two-table join query with correlated predicates and verifies the AST structure.'

9. PHASE 6: DEEP LEARNING PIPELINE

9.1 Architecture Overview

PREDICATE leaf → Column embedding + Operator embedding + Value embedding + BN probability lookup → Concatenate → leaf vector (128-dim) → Tree-LSTM (Child-Sum with AND/OR/JOIN gates) → Query embedding (128-dim) → Concatenate with FILTERED BN probability vector → MLP (256→128→64→1) → Predicted log(cardinality)

9.2 Key Design: FILTERED BN Vector (Point 3 Fix)

Instead of concatenating ALL possible BN probabilities (hundreds of dimensions), filter to only those relevant to predicates in THIS query.

```
import torch

def get_relevant_bn_probabilities(
    query_ast,
    bn_models,
    join_bridge=None,
    max_facts=10
):
    probs = []

    def collect_predicates(node):
        if node.node_type == "PREDICATE":
            return [node.value]

        result = []

        for child in node.children:
            result.extend(
                collect_predicates(child)
            )

        return result

    predicates = collect_predicates(query_ast)

    for pred in predicates:
        if not pred:
            continue

        table = pred.get("table")
        column = pred.get("column")
        value = pred.get("value")

        if not table or not column:
            continue

        column = column.split(".")[1]

        # -----
        # Single-table BN probability
        # -----
        if table in bn_models:
```

```

evidence = {}

for other_pred in predicates:

    if other_pred is pred:
        continue

    if other_pred.get("table") == table:

        other_column = (
            other_pred["column"]
            .split(".")[-1]
        )

        evidence[other_column] = (
            other_pred["value"]
        )

    try:

        prob = bn_models[table].query_probability(
            evidence
        )

        probs.append(float(prob))

    except Exception:

        probs.append(0.5)

# -----
# Cross-table join bridge
# -----
if join_bridge is not None:

    tables = {
        p.get("table")
        for p in predicates
        if p.get("table")
    }

    if "employees" in tables and "projects" in tables:

        employee_preds = [
            p for p in predicates
            if p.get("table") == "employees"
        ]

        project_preds = [
            p for p in predicates
            if p.get("table") == "projects"
        ]

        if employee_preds and project_preds:

            try:

                employee_pred = employee_preds[0]
                project_pred = project_preds[0]

                join_prob = (
                    join_bridge.query_probability(
                        join_val="Engineering",
                        t1_predicate={
                            employee_pred["column"]

```

```

        .split(".")[1]:
        employee_pred["value"]
    },
    t2_predicate={
        project_pred["column"]
        .split(".")[1]:
        project_pred["value"]
    }
    )
)

probs.append(
    float(join_prob)
)

except Exception:

    probs.append(0.5)

# Fixed-size fact vector.
probs = probs[:max_facts]

probs += [0.0] * (
    max_facts - len(probs)
)

return torch.tensor(
    probs,
    dtype=torch.float32
)

```

9.3 Tree-LSTM Implementation

```

import torch
import torch.nn as nn

class ChildSumTreeLSTM(nn.Module):

    def __init__(
        self,
        embedding_dim: int,
        hidden_dim: int
    ):
        super().__init__()

        self.embedding_dim = embedding_dim
        self.hidden_dim = hidden_dim

        self.and_weights = self._create_gate_weights()
        self.or_weights = self._create_gate_weights()
        self.join_weights = self._create_gate_weights()

        self.leaf_encoder = nn.Sequential(
            nn.Linear(
                embedding_dim,
                hidden_dim
            ),
            nn.ReLU(),
            nn.Linear(
                hidden_dim,
                hidden_dim
            )
        )
    )

```

```

def _create_gate_weights(self):

    return nn.ModuleDict({

        "W_i": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "W_f": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "W_o": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "W_u": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "U_i": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "U_f": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "U_o": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        ),

        "U_u": nn.Linear(
            self.hidden_dim,
            self.hidden_dim
        )
    })

def forward(
    self,
    node,
    node_type=None
):

    if node.node_type == "PREDICATE":

        leaf_vec = self.leaf_encoder(
            node.leaf_vector
        )

        cell_state = torch.zeros_like(
            leaf_vec
        )

        return leaf_vec, cell_state

    node_type = node.node_type

```

```

if node_type not in {
    "AND",
    "OR",
    "JOIN"
}:
    raise ValueError(
        f"Unsupported node type: {node_type}"
    )

weights = {
    "AND": self.and_weights,
    "OR": self.or_weights,
    "JOIN": self.join_weights
}[node_type]

child_h = []
child_c = []

for child in node.children:

    h, c = self.forward(
        child,
        child.node_type
    )

    child_h.append(h)
    child_c.append(c)

if not child_h:
    raise ValueError(
        f"{node_type} node has no children."
    )

h_sum = torch.stack(
    child_h
).sum(dim=0)

c_sum = torch.stack(
    child_c
).sum(dim=0)

i = torch.sigmoid(
    weights["w_i"](h_sum)
    + weights["u_i"](c_sum)
)

o = torch.sigmoid(
    weights["w_o"](h_sum)
    + weights["u_o"](c_sum)
)

f_gates = [

    torch.sigmoid(
        weights["w_f"](h)
        + weights["u_f"](c)
    )

    for h, c in zip(
        child_h,
        child_c
    )
]

u = torch.tanh(

```

```

        weights["W_u"](h_sum)
        + weights["U_u"](c_sum)
    )

    c_new = (
        i * u
        + sum(
            f * c
            for f, c in zip(
                f_gates,
                child_c
            )
        )
    )

    h_new = (
        o
        * torch.tanh(c_new)
    )

    return h_new, c_new

```

9.4 Fusion MLP

```

import torch
import torch.nn as nn

class FusionMLP(nn.Module):

    def __init__(
        self,
        query_embed_dim: int,
        fact_dim: int,
        hidden_dims: list,
        output_dim: int = 1
    ):

        super().__init__()

        input_dim = (
            query_embed_dim
            + fact_dim
        )

        layers = []
        prev_dim = input_dim

        for h_dim in hidden_dims:

            layers.extend([

                nn.Linear(
                    prev_dim,
                    h_dim
                ),

                nn.ReLU(),

                nn.Dropout(0.2)
            ])

            prev_dim = h_dim

        layers.append(

```

```

        nn.Linear(
            prev_dim,
            output_dim
        )
    )

    self.mlp = nn.Sequential(
        *layers
    )

def forward(
    self,
    query_embedding: torch.Tensor,
    fact_vector: torch.Tensor
) -> torch.Tensor:

    combined = torch.cat(
        [
            query_embedding,
            fact_vector
        ],
        dim=-1
    )

    return self.mlp(
        combined
    )

```

GOOGLE AI STUDIO PROMPT for Phase 6:

'Implement the complete deep learning pipeline for cardinality estimation: (1) Leaf encoder that combines column/operator/value embeddings with BN probability lookup, (2) Child-Sum Tree-LSTM with different weights for AND/OR/JOIN nodes, (3) Fusion MLP that takes the 128-dim query embedding and a FILTERED fact vector containing only BN probabilities relevant to predicates in this query, padded to fixed length 10. The MLP outputs log(cardinality). Include gradient norm monitoring to verify both the Tree-LSTM branch and BN fact branch are being used during training. CRITICAL: The Tree-LSTM never sees raw probability numbers — only structural embeddings.'

10. PHASE 7: TRAINING LOOP

10.1 Loss Function

```
import torch
import torch.nn as nn

class LogMSELoss(nn.Module):

    def __init__(self):
        super().__init__()
        self.mse = nn.MSELoss()

    def forward(
        self,
        pred_log_card,
        true_card
    ):

        true_log_card = torch.log(
            true_card.float() + 1.0
        )

        return self.mse(
            pred_log_card.squeeze(),
            true_log_card.squeeze()
        )

class QErrorLoss(nn.Module):

    def __init__(self):
        super().__init__()

    def forward(
        self,
        pred_card,
        true_card
    ):

        # Avoid division by zero.
        pred_card = torch.clamp(
            pred_card,
            min=1e-8
        )

        true_card = torch.clamp(
            true_card,
            min=1e-8
        )

        ratio1 = (
            pred_card
            / true_card
        )

        ratio2 = (
            true_card
            / pred_card
        )
```

```

        q_error = torch.maximum(
            ratio1,
            ratio2
        )

    return torch.mean(
        q_error
    )

```

10.2 Training Loop with Gradient Monitoring

```

import os
import numpy as np
import torch

from torch.utils.data import DataLoader
from tqdm import tqdm

try:
    import wandb
except ImportError:
    wandb = None

class Trainer:

    def __init__(
        self,
        model,
        optimizer,
        config,
        criterion=None,
        scheduler=None
    ):

        self.model = model
        self.optimizer = optimizer
        self.config = config

        self.criterion = (
            criterion
            if criterion is not None
            else LogMSELoss()
        )

        self.scheduler = scheduler

        self.best_val_metric = float("inf")
        self.patience_counter = 0

    def train_epoch(
        self,
        train_loader: DataLoader
    ):

        self.model.train()

        total_loss = 0.0

        for batch in tqdm(
            train_loader,
            desc="Training"
        ):

```

```

self.optimizer.zero_grad()

pred_log_card = self.model(
    batch["ast"],
    batch["bn_facts"]
)

true_card = (
    batch["true_cardinality"]
)

loss = self.criterion(
    pred_log_card,
    true_card
)

loss.backward()

# -----
# Gradient norm monitoring
# -----

tree_lstm_grad_norm = 0.0
fusion_grad_norm = 0.0

for name, param in (
    self.model.named_parameters()
):

    if param.grad is None:
        continue

    grad_norm = (
        param.grad.norm().item()
    )

    if "tree_lstm" in name:
        tree_lstm_grad_norm += (
            grad_norm
        )

    elif "fusion" in name:
        fusion_grad_norm += (
            grad_norm
        )

grad_ratio = (
    tree_lstm_grad_norm
    / (fusion_grad_norm + 1e-8)
)

if wandb is not None:

    wandb.log({

        "train/loss": loss.item(),

        "grad/tree_lstm_norm":
            tree_lstm_grad_norm,

        "grad/fusion_norm":
            fusion_grad_norm,

        "grad/tree_to_fusion_ratio":
            grad_ratio
    })

```

```

    })

    if tree_lstm_grad_norm < 1e-6:

        print(
            "WARNING: Tree-LSTM gradients "
            "near zero!"
        )

    if fusion_grad_norm < 1e-6:

        print(
            "WARNING: Fusion MLP gradients "
            "near zero!"
        )

    self.optimizer.step()

    total_loss += (
        loss.item()
    )

    return (
        total_loss
        / max(len(train_loader), 1)
    )

def validate(
    self,
    val_loader: DataLoader
):

    self.model.eval()

    q_errors = []

    with torch.no_grad():

        for batch in val_loader:

            pred_log_card = self.model(
                batch["ast"],
                batch["bn_facts"]
            )

            # Convert log(cardinality)
            # back to cardinality.
            pred_card = torch.exp(
                pred_log_card
            ).squeeze()

            true_card = (
                batch["true_cardinality"]
                .float()
                .squeeze()
            )

            pred_card = torch.clamp(
                pred_card,
                min=1e-8
            )

            true_card = torch.clamp(
                true_card,
                min=1e-8
            )

```

```

        )

        ratio1 = (
            pred_card
            / true_card
        )

        ratio2 = (
            true_card
            / pred_card
        )

        q_error = torch.maximum(
            ratio1,
            ratio2
        )

        q_errors.extend(
            q_error.detach()
                .cpu()
                .numpy()
                .reshape(-1)
                .tolist()
        )

    if not q_errors:
        return {
            "median_q_error": float("inf"),
            "p90_q_error": float("inf"),
            "p99_q_error": float("inf")
        }

    median_q = np.median(
        q_errors
    )

    p90_q = np.percentile(
        q_errors,
        90
    )

    p99_q = np.percentile(
        q_errors,
        99
    )

    return {
        "median_q_error": median_q,
        "p90_q_error": p90_q,
        "p99_q_error": p99_q
    }

def train(
    self,
    train_loader: DataLoader,
    val_loader: DataLoader,
    num_epochs: int
):
    patience = (
        self.config
        .get("model", {})
        .get("training", {})
        .get("patience", 15)
    )

```

```

checkpoint_dir = (
    self.config
    .get("project", {})
    .get("root_dir", ".")
)

checkpoint_dir = os.path.join(
    checkpoint_dir,
    "checkpoints"
)

os.makedirs(
    checkpoint_dir,
    exist_ok=True
)

checkpoint_path = os.path.join(
    checkpoint_dir,
    "best_model.pt"
)

for epoch in range(
    num_epochs
):

    train_loss = (
        self.train_epoch(
            train_loader
        )
    )

    val_metrics = (
        self.validate(
            val_loader
        )
    )

    median_q = (
        val_metrics[
            "median_q_error"
        ]
    )

    print(
        f"Epoch {epoch + 1}: "
        f"train_loss={train_loss:.4f}, "
        f"val_median_q={median_q:.3f}"
    )

    if wandb is not None:

        wandb.log({

            "epoch": epoch + 1,

            "val/median_q_error":
                median_q,

            "val/p90_q_error":
                val_metrics[
                    "p90_q_error"
                ],

            "val/p99_q_error":
                val_metrics[

```

```

        "p99_q_error"
    ]
})

# -----
# Learning-rate scheduler
# -----

if self.scheduler is not None:

    # ReduceLROnPlateau expects
    # the monitored metric.
    self.scheduler.step(
        median_q
    )

# -----
# Best checkpoint
# -----

if median_q < (
    self.best_val_metric
):

    self.best_val_metric = (
        median_q
    )

    self.patience_counter = 0

    torch.save(
        self.model.state_dict(),
        checkpoint_path
    )

    print(
        f"Saved best model → "
        f"{checkpoint_path}"
    )

else:

    self.patience_counter += 1

    if (
        self.patience_counter
        >= patience
    ):

        print(
            f"Early stopping at "
            f"epoch {epoch + 1}"
        )

        break

```

10.3 ReduceLROnPlateau Example

```

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-3,
    weight_decay=1e-5
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(

```

```

optimizer,
mode="min",
factor=0.5,
patience=5,
min_lr=1e-6
)

trainer = Trainer(
    model=model,
    optimizer=optimizer,
    config=config,
    criterion=LogMSELoss(),
    scheduler=scheduler
)

```

GOOGLE AI STUDIO PROMPT for Phase 7:

'Implement the complete training loop for the Tree-LSTM + Fusion MLP cardinality estimator. Use MSE loss in log-space. Track validation median Q-error every epoch. Implement early stopping with patience=15. CRITICAL: Include gradient norm monitoring that logs separate gradient norms for the Tree-LSTM branch vs the fusion MLP branch. Alert if either branch has near-zero gradients. Save the best checkpoint by validation median Q-error. Use W&B; for experiment tracking. Include a ReduceLROnPlateau learning-rate scheduler.'